

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
Факультет физико-математических и естественных наук

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5

Студент: Николаева Ангелина Борисовна

Студенческий билет: 1032253612

Группа: НКАбд-04-25

МОСКВА

2026 г

Содержание

1. Цель работы	3
2. Выполнение лабораторной работы	4
2.1. Основы работы с Midnight Commander	4
2.2. Работа в NASM	5
2.3. Подключение внешнего файла	5
2.4. Задание для самостоятельной работы.....	8
2.5. Контрольные вопросы для самопроверки	12
3. Вывод	14

1. Цель работы.

Целью данной лабораторной работы является приобретение практических навыков работы в Midnight Commander, освоение инструкций языка ассемблера mov и int.

2. Выполнение лабораторной работы

2.1. Основы работы с Midnight Commander

Введя соответствующую команду в терминале (mc) я открываю Midnight Commander (рис.1)

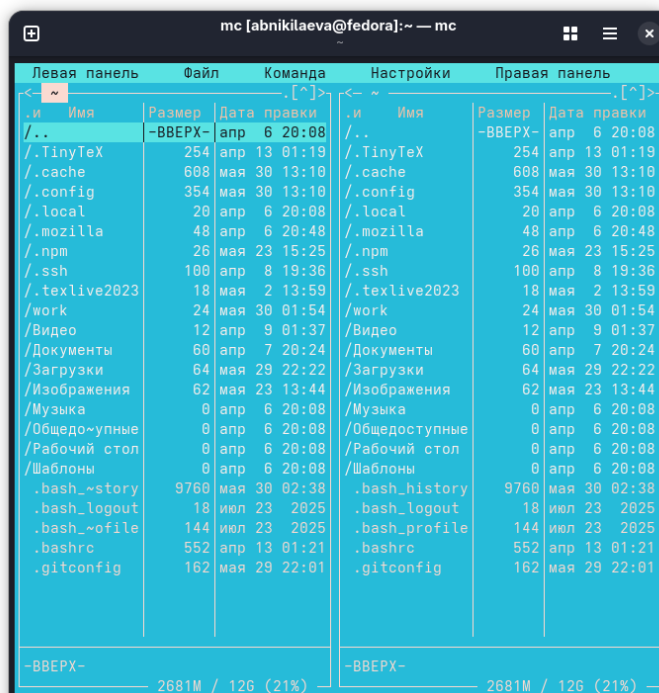


Рис. 1 : Интерфейс Midnight Commander

Перехожу в созданный каталог в предыдущей лабораторной работе (рис. 2)

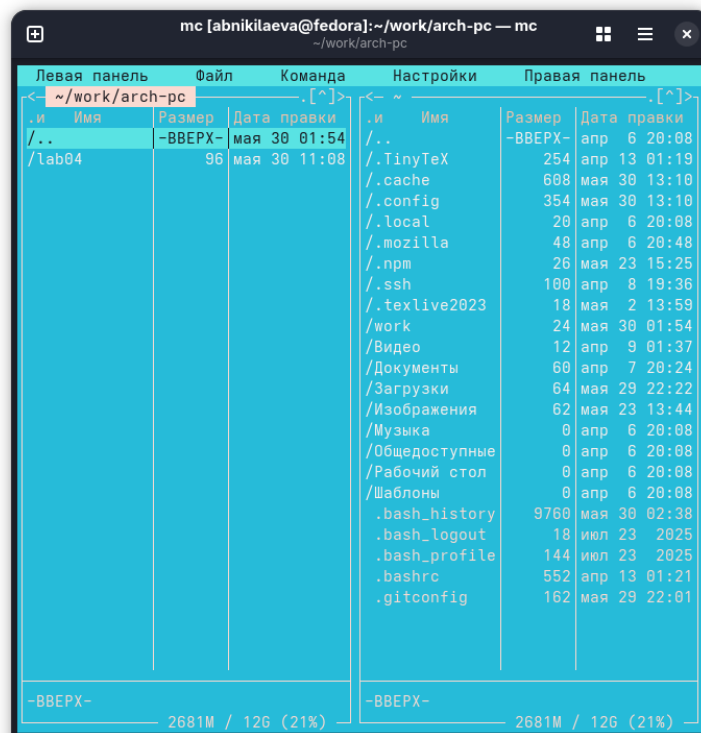


Рис. 2

С помощью функциональной клавиши, я создаю подкаталог lab05, в котором буду работать (рис. 3)

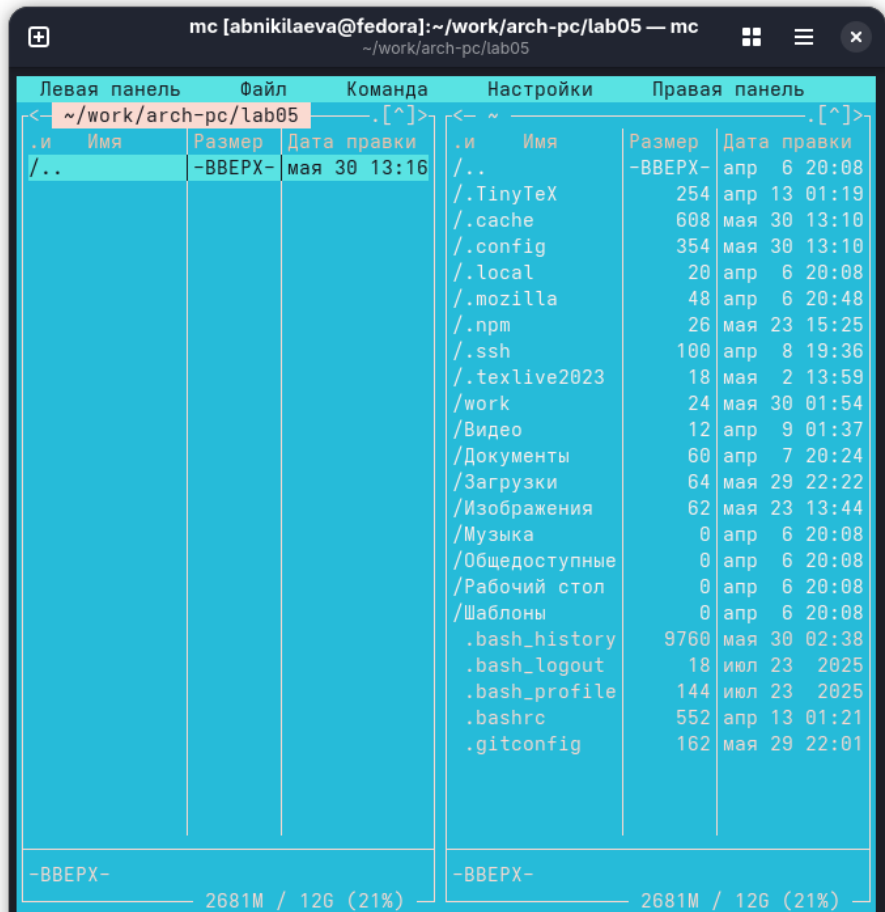


Рис. 3

В строке ввода вводжу команду touch и создаю файл (рис. 4)

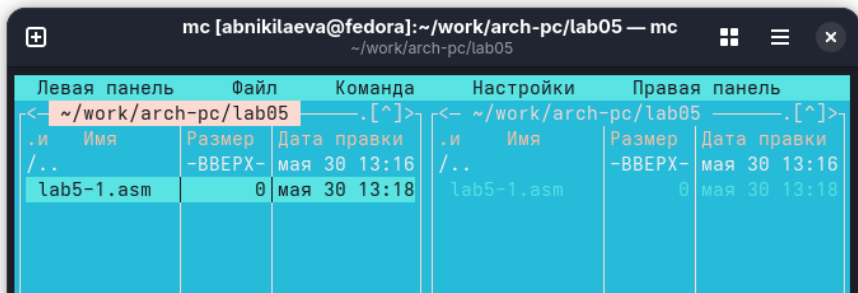
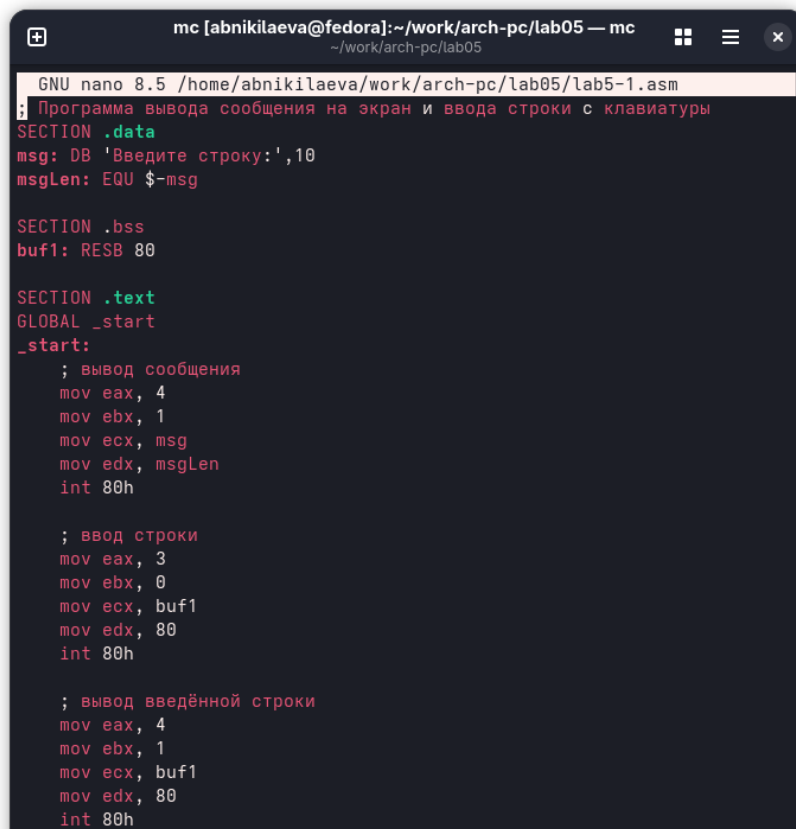


Рис. 4

С помощью F4 открываю только что созданный файл и вношу код с листинга (рис.5)

2.2. Работа в NASM.

С помощью F4 открываю только что созданный файл и вношу код с листинга (рис. 5)



```
mc [abnikilaeva@fedora]:~/work/arch-pc/lab05 — mc
~/work/arch-pc/lab05
GNU nano 8.5 /home/abnikilaeva/work/arch-pc/lab05/lab5-1.asm
; Программа вывода сообщения на экран и ввода строки с клавиатуры
SECTION .data
msg: DB 'Введите строку:',10
msgLen: EQU $-msg

SECTION .bss
buf1: RESB 80

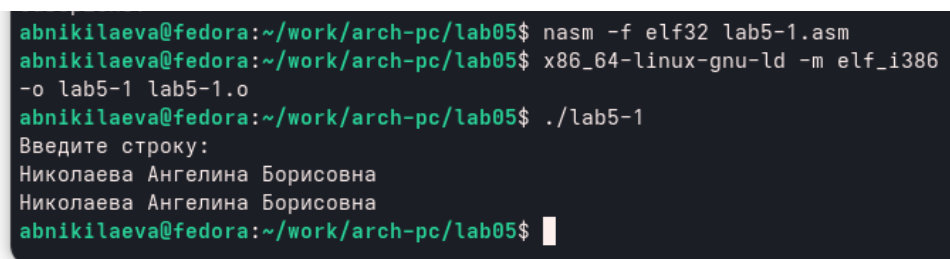
SECTION .text
GLOBAL _start
_start:
; вывод сообщения
mov eax, 4
mov ebx, 1
mov ecx, msg
mov edx, msgLen
int 80h

; ввод строки
mov eax, 3
mov ebx, 0
mov ecx, buf1
mov edx, 80
int 80h

; вывод введенной строки
mov eax, 4
mov ebx, 1
mov ecx, buf1
mov edx, 80
int 80h
```

Рис. 5

Проверяю сохраненные изменения с помощью клавиши F3. Транслирую и компоную измененный файл, запускаю (рис. 6)



```
abnikilaeva@fedora:~/work/arch-pc/lab05$ nasm -f elf32 lab5-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab05$ x86_64-linux-gnu-ld -m elf_i386
-o lab5-1 lab5-1.o
abnikilaeva@fedora:~/work/arch-pc/lab05$ ./lab5-1
Введите строку:
Николаева Ангелина Борисовна
Николаева Ангелина Борисовна
abnikilaeva@fedora:~/work/arch-pc/lab05$
```

Рис. 6: Трансляция, компоновка и последующий запуск программ

2.3. Подключение внешнего файла.

Скачанный с ТУИС файл сохраняю в общую папку на своем компьютере, на виртуальной машине в интерфейсе Midnight Commander перехожу в директорию общей папки, копирую файл в рабочий подкаталог. (рис. 7)



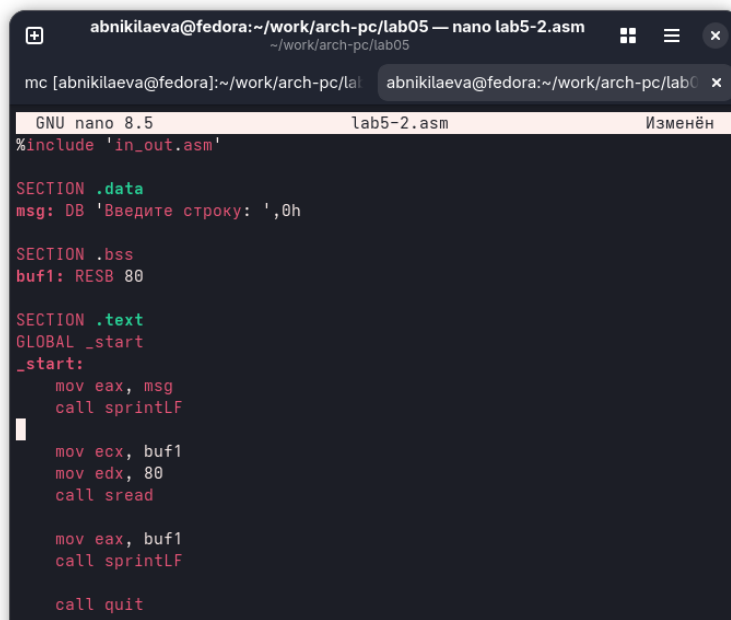
Рис.7: Копирование файла в рабочий каталог

Создаю копию файла для последующей работы с ним (рис. 8)



Рис. 8: Создание копии файла в Midnight Commander

В копии файла подключаю подпрограмму из подключенного файла (рис.9)



```
GNU nano 8.5 lab5-2.asm Изменён
mc [abnikilaeva@fedora]:~/work/arch-pc/lab05 abnikilaeva@fedora:~/work/arch-pc/lab05

#include 'in_out.asm'

SECTION .data
msg: DB 'Введите строку: ',0h

SECTION .bss
buf1: RESB 80

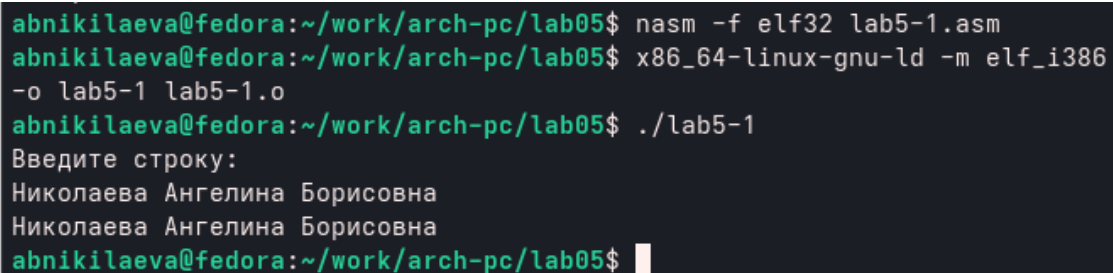
SECTION .text
GLOBAL _start
_start:
    mov eax, msg
    call sprintf

    mov ecx, buf1
    mov edx, 80
    call sread

    mov eax, buf1
    call sprintf
    call quit
```

Рис. 9: Изменение программы

Транслирую, компоную и запускаю программу с подключенным файлом (рис.10)

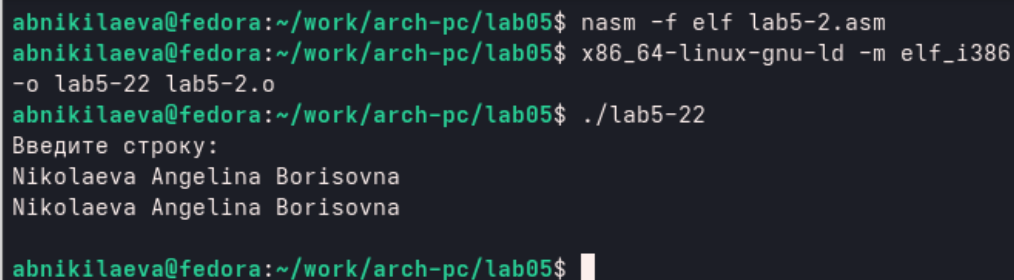


```
abnikilaeva@fedora:~/work/arch-pc/lab05$ nasm -f elf32 lab5-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab05$ x86_64-linux-gnu-ld -m elf_i386
-o lab5-1 lab5-1.o
abnikilaeva@fedora:~/work/arch-pc/lab05$ ./lab5-1
Введите строку:
Николаева Ангелина Борисовна
Николаева Ангелина Борисовна
abnikilaeva@fedora:~/work/arch-pc/lab05$
```

Рис. 10: Запуск измененной программы

Редактирую файл и заменяю в нем подпрограмму sprintf на sprint. Разница подпрограмм в том, что вторая вызывает ввод на той же строке(рис. 11)

2.4. Задание для самостоятельной работы.

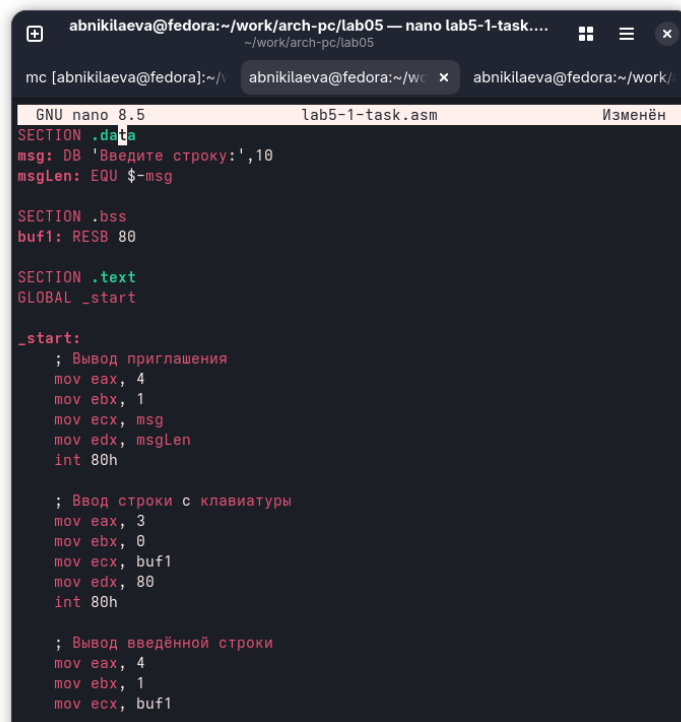


```
abnikilaeva@fedora:~/work/arch-pc/lab05$ nasm -f elf lab5-2.asm
abnikilaeva@fedora:~/work/arch-pc/lab05$ x86_64-linux-gnu-ld -m elf_i386
-o lab5-22 lab5-2.o
abnikilaeva@fedora:~/work/arch-pc/lab05$ ./lab5-22
Введите строку:
Nikolaeva Angelina Borisovna
Nikolaeva Angelina Borisovna
abnikilaeva@fedora:~/work/arch-pc/lab05$
```

Рис. 11

Создаю копию lab5-1.asm, редактирую так, чтобы в конце выводилась введенная

мною строка с клавиатуры (рис. 12)



```
GNU nano 8.5 lab5-1-task.asm Изменён
mc [abnikilaeva@fedora]:~/ / abnikilaeva@fedora:~/work/
SECTION .data
msg: DB 'Введите строку:',10
msgLen: EQU $-msg

SECTION .bss
buf1: RESB 80

SECTION .text
GLOBAL _start

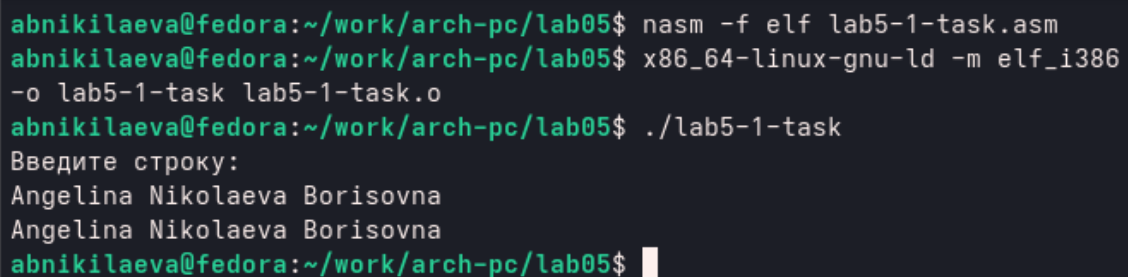
_start:
; Вывод приглашения
mov eax, 4
mov ebx, 1
mov ecx, msg
mov edx, msgLen
int 80h

; Ввод строки с клавиатуры
mov eax, 3
mov ebx, 0
mov ecx, buf1
mov edx, 80
int 80h

; Вывод введенной строки
mov eax, 4
mov ebx, 1
mov ecx, buf1
```

Рис. 12

Транслирую, компоную и запускаю свою программу (рис.13)



```
abnikilaeva@fedora:~/work/arch-pc/lab05$ nasm -f elf lab5-1-task.asm
abnikilaeva@fedora:~/work/arch-pc/lab05$ x86_64-linux-gnu-ld -m elf_i386
-o lab5-1-task lab5-1-task.o
abnikilaeva@fedora:~/work/arch-pc/lab05$ ./lab5-1-task
Введите строку:
Angelina Nikolaeva Borisovna
Angelina Nikolaeva Borisovna
abnikilaeva@fedora:~/work/arch-pc/lab05$
```

Рис. 13

Код прикладываю

```
SECTION .data
msg: DB 'Введите строку:',10
msgLen: EQU $-msg
```

```
SECTION .bss
buf1: RESB 80
```

```
SECTION .text
GLOBAL _start
```

```
_start:
; Вывод приглашения
mov eax, 4
```

```
mov ebx, 1
mov ecx, msg
mov edx, msgLen
int 80h
```

; Ввод строки с клавиатуры

```
mov eax, 3
mov ebx, 0
mov ecx, buf1
mov edx, 80
int 80h
```

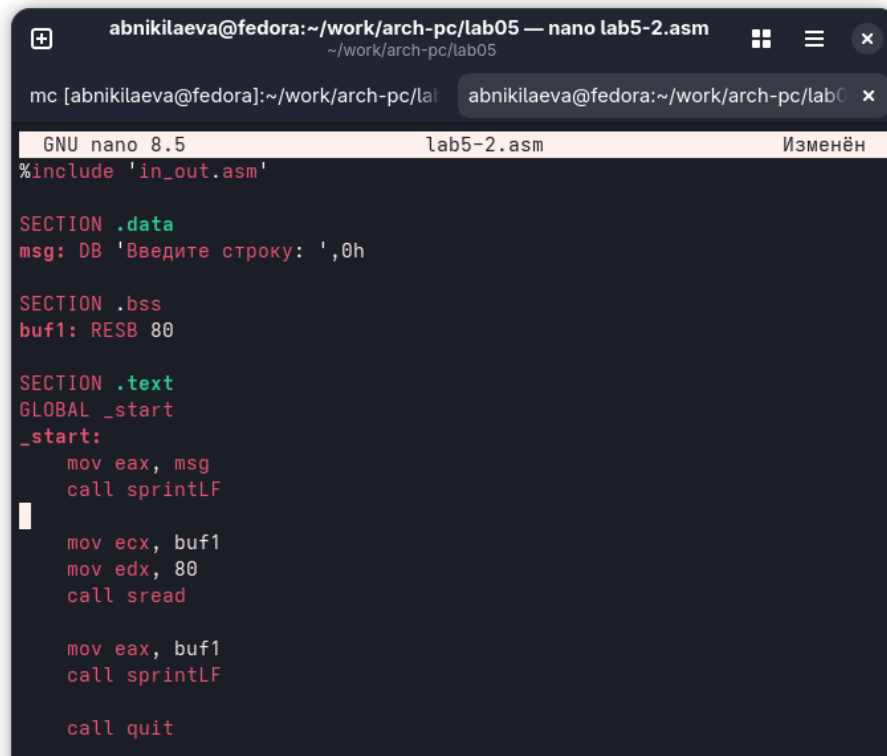
; Вывод введенной строки

```
mov eax, 4
mov ebx, 1
mov ecx, buf1
mov edx, 80
int 80h
```

; Выход из программы

```
mov eax, 1
mov ebx, 0
int 80h
```

Создаю копию lab5-2.asm, редактирую так, чтобы в конце выводилась введенная мною строка с клавиатуры (рис. 14)



```
abnikilaeva@fedora:~/work/arch-pc/lab05 — nano lab5-2.asm
~/work/arch-pc/lab05
mc [abnikilaeva@fedora]:~/work/arch-pc/lab05 abnikilaeva@fedora:~/work/arch-pc/lab05
GNU nano 8.5 lab5-2.asm Изменён
#include 'in_out.asm'

SECTION .data
msg: DB 'Введите строку: ',0h

SECTION .bss
buf1: RESB 80

SECTION .text
GLOBAL _start
_start:
    mov eax, msg
    call sprintLF

    mov ecx, buf1
    mov edx, 80
    call sread

    mov eax, buf1
    call sprintLF

    call quit
```

Рис. 14

Транслирую, компоную и запускаю свою программу (рис.15)

```

abnikilaeva@fedora:~/work/arch-pc/lab05$ nasm -f elf lab5-2-task.asm
abnikilaeva@fedora:~/work/arch-pc/lab05$ x86_64-linux-gnu-ld -m elf_i386
-o lab5-2-task lab5-2-task.o
abnikilaeva@fedora:~/work/arch-pc/lab05$ ./lab5-2-task
Введите строку: Nikolaeva Angelina
Nikolaeva Angelina
abnikilaeva@fedora:~/work/arch-pc/lab05$ █

```

Рис. 15

Код прикладываю:

```
%include 'in_out.asm'
```

```
SECTION .data
```

```
msg: DB 'Введите строку: ', 0h
```

```
SECTION .bss
```

```
buf1: RESB 80
```

```
SECTION .text
```

```
GLOBAL _start
```

```
_start:
```

```
    mov eax, msg
```

```
    call sprint      ; вывод приглашения
```

```
    mov ecx, buf1
```

```
    mov edx, 80
```

```
    call sread      ; ввод строки
```

```
    mov eax, buf1
```

```
    call sprint      ; вывод введённой строки
```

```
    call quit
```

2.5. Контрольные вопросы для самопроверки.

1. Каково назначение mc?

Midnight Commander (mc) — это консольный файловый менеджер, предназначенный для просмотра структуры каталогов и выполнения основных операций по управлению файловой системой. Он позволяет наглядно и удобно работать с файлами, копировать, перемещать, удалять, переименовывать файлы и каталоги, редактировать файлы во встроенном редакторе, а также выполнять команды оболочки, не выходя из программы.

2. Какие операции с файлами можно выполнить как с помощью команд `bash`, так и с помощью меню (комбинаций клавиш) `mc`? Приведите несколько примеров.

Одни и те же операции можно выполнять как стандартными командами `bash`, так и функциональными клавишами в `mc`. Например, копирование файла можно выполнить командой `cp` в `bash` или клавишей F5 в `mc`; перемещение или переименование — командой `mv` или клавишей F6; удаление — командой `rm` или клавишей F8; создание подкаталога — командой `mkdir` или клавишей F7; просмотр содержимого файла — командой `cat` или `less`, либо клавишей F3; редактирование файла — через `nano`, `vim` или клавишей F4.

3. Какова структура программы на языке ассемблера NASM?

Программа на ассемблере NASM обычно состоит из трёх основных секций: секция `.data` (инициализированных данных) для переменных, которым задано начальное значение; секция `.bss` (неинициализированных данных) для переменных, под которые только выделяется память, а значение присваивается в ходе выполнения программы; и секция `.text` (кода программы), которая содержит исполняемые инструкции. Обязательным элементом является объявление глобальной метки `_start`, которая является точкой входа в программу, и завершающий программу системный вызов `exit`.

4. Для описания каких данных используются секции `bss` и `data` в языке ассемблера NASM?

Секция `.data` используется для описания инициализированных данных — то есть переменных, значения которых известны на этапе компиляции и записываются в исполняемый файл. Секция `.bss` используется для описания неинициализированных данных — переменных, под которые во время компиляции только резервируется память (без указания начальных значений), а фактические значения присваиваются уже во время выполнения программы.

5. Для чего используются компоненты `db`, `dw`, `dd`, `dq` и `dt` языка ассемблера NASM?

Эти директивы используются в секции `.data` для резервирования памяти и указания начальных значений переменных. `DB` (define byte) определяет переменную размером 1 байт и используется чаще всего для хранения строк и символов. `DW` (define word) определяет переменную размером 2 байта (слово). `DD` (define double word) определяет переменную размером 4 байта (двойное слово). `DQ` (define quad word) определяет

переменную размером 8 байт (учетверённое слово). DT (define ten bytes) определяет переменную размером 10 байт, обычно используется для чисел с плавающей точкой.

6. Какое произойдёт действие при выполнении инструкции `mov eax, esi`?

При выполнении инструкции `mov eax, esi` происходит копирование (пересылка) значения из регистра-источника `esi` в регистр-приёмник `eax`. При этом содержимое регистра `esi` остаётся неизменным, а в регистр `eax` записывается точная копия того значения, которое хранилось в `esi`. Например, если до выполнения инструкции в `esi` было число 100, а в `eax` было число 50, то после выполнения `mov eax, esi` в обоих регистрах будет число 100.

7. Для чего используется инструкция `int 80h`?

Инструкция `int 80h` предназначена для вызова программного прерывания с номером 80h (в шестнадцатеричной системе), что в операционной системе Linux соответствует обращению к ядру для выполнения системного вызова. Перед вызовом `int 80h` в регистр `eax` необходимо поместить номер требуемой системной функции (например, 1 для выхода из программы, 3 для чтения с клавиатуры, 4 для вывода на экран), а в регистры `ebx`, `ecx`, `edx` — соответствующие параметры. После выполнения системного вызова результат возвращается в регистр `eax`. Таким образом, `int 80h` является основным способом взаимодействия программы с ядром операционной системы.

3.Вывод

При выполнении данной лабораторной работы я приобрела практические навыки работы в Midnight Commander, а также освоил инструкции языка ассемблера mov и int.